

EGEC 441-01 SP'26

Final Project

32-Bit Adder, Verilog HDL

James Hoang

May 15, 2026

Introduction

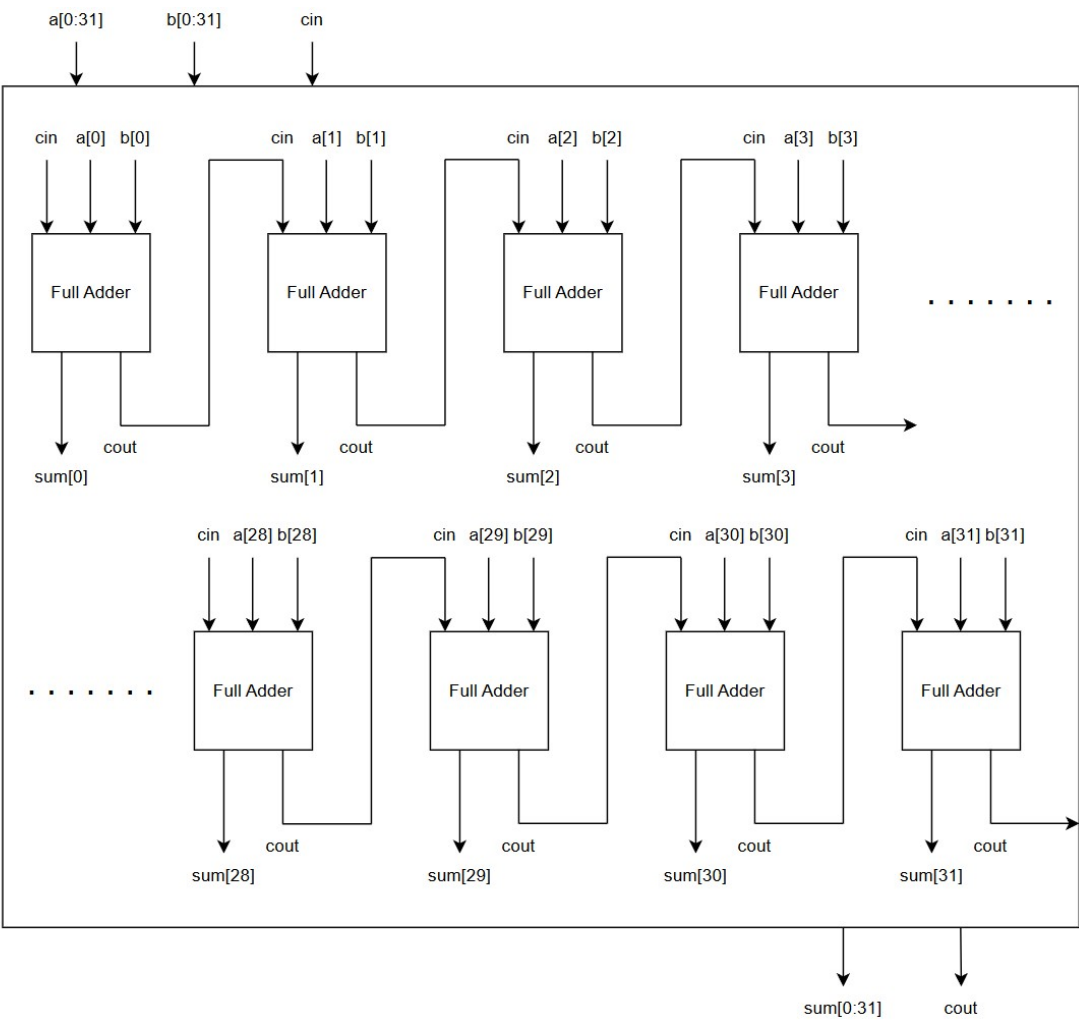
This project focuses on the design, verification, and physical implementation of a 32-bit ripple-carry adder using Verilog HDL and the OpenLane digital design flow. Adders are fundamental arithmetic circuits used in processors, ALUs, counters, memory address calculations, and many other digital systems. For this project, the 32-bit adder was selected because it is simple enough to verify clearly, but large enough to demonstrate meaningful RTL design, standard-cell synthesis, physical placement, routing, and final layout generation.

The adder was implemented structurally using a 1-bit full-adder module repeated across 32 stages. Each full adder receives one bit from the two input operands along with a carry-in signal, then produces a sum bit and carry-out signal. The carry-out of each stage connects to the carry-in of the next stage, creating a ripple-carry structure. This design demonstrates modular Verilog coding, parameterized hardware design, and the relationship between logical RTL implementation and physical chip layout.

After the Verilog design was completed, a testbench was used to verify correct functionality using several input cases, including simple addition and carry-out/overflow conditions. The verified RTL design was then passed through the OpenLane RTL-to-GDS flow using the SKY130 standard-cell library. The flow included synthesis, floorplanning, tap/endcap insertion, I/O placement, power distribution network generation, placement, routing, static timing analysis, and final GDS layout generation. Through this process, the project demonstrates how a Verilog hardware description can be transformed into a physical chip-level implementation.

Design Description

i. Block Diagram



ii. Truth Table (for each 1-bit Full Adder)

a	0	0	0	0	1	1	1	1
b	0	0	1	1	0	0	1	1
cin	0	1	0	1	0	1	0	1
sum	0	1	1	0	1	0	0	1
cout	0	0	0	1	0	1	1	1

iii. Program-Implemented Equations

```
sum = a ^ b ^ cin;  
cout = (a & b) | (a & cin) | (b & cin);
```

Verilog Implementation

The following circuit was implemented into Verilog as a parameterized ripple-carry adder; default size is 32-bits, but it can be changed to any number. Structurally, it connects multiple 1-bit full adders by calling a separate 'full_adder' module.

- full_adder Module:
 - Inputs: a, b, cin (1-bit)
 - Outputs: sum, cout (1-bit)
- adder Module:
 - Inputs: a, b (variable width, 32-bits in this case), cin (1-bit)
 - Outputs: sum (32-bit), cout (1-bit)
- Internal Signal: carry (1-bit carry out/in between Full Adders)

```

module full_adder (input a, input b, input cin, output sum,
output cout);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);
endmodule

module adder #(parameter WIDTH = 32) (
    input [WIDTH-1:0] a,
    input [WIDTH-1:0] b,
    input cin,
    output [WIDTH-1:0] sum,
    output cout
);

    wire [WIDTH:0] carry;
    assign carry[0] = cin;

    genvar i;
    generate
        for (i = 0; i < WIDTH; i = i + 1) begin: adder_loop
            full_adder fa (
                .a(a[i]),
                .b(b[i]),
                .cin(carry[i]),
                .sum(sum[i]),
                .cout(carry[i+1])
            );
        end
    endgenerate
    assign cout = carry[WIDTH];

endmodule

```

Functional Verification**TB.v (Testbench)**

```

module tb_adder;

    reg [31:0] a;
    reg [31:0] b;
    reg cin;
    wire [31:0] sum;
    wire cout;

    adder dut (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

    initial begin
        $dumpfile("waveform.vcd");
        $dumpvars(0, tb_adder);

        // Test case 1
        a = 32'h00000001; b = 32'h00000001; cin = 0; #10;
        if (sum !== 32'h00000002 || cout !== 0) begin
            $display("TB_FAIL: Test case 1 failed"); $finish; end

        // Test case 2
        a = 32'hFFFFFFFF; b = 32'h00000001; cin = 0; #10;
        if (sum !== 32'h00000000 || cout !== 1) begin
            $display("TB_FAIL: Test case 2 failed"); $finish; end

        // Test case 3
        a = 32'h12345678; b = 32'h87654321; cin = 0; #10;
        if (sum !== 32'h99999999 || cout !== 0) begin
            $display("TB_FAIL: Test case 3 failed"); $finish; end

        // Test case 4
        a = 32'hFFFFFFFF; b = 32'hFFFFFFFF; cin = 1; #10;
        if (sum !== 32'hFFFFFFFF || cout !== 1) begin
            $display("TB_FAIL: Test case 4 failed"); $finish; end

        // Test case 5 (added to verify carry out)
        a = 32'hFFFFFFFF; b = 32'hFFFFFFFF; cin = 0; #10;
        if (sum !== 32'hFFFFFFFE || cout !== 1) begin
            $display("TB_FAIL: Test case 5 failed"); $finish; end

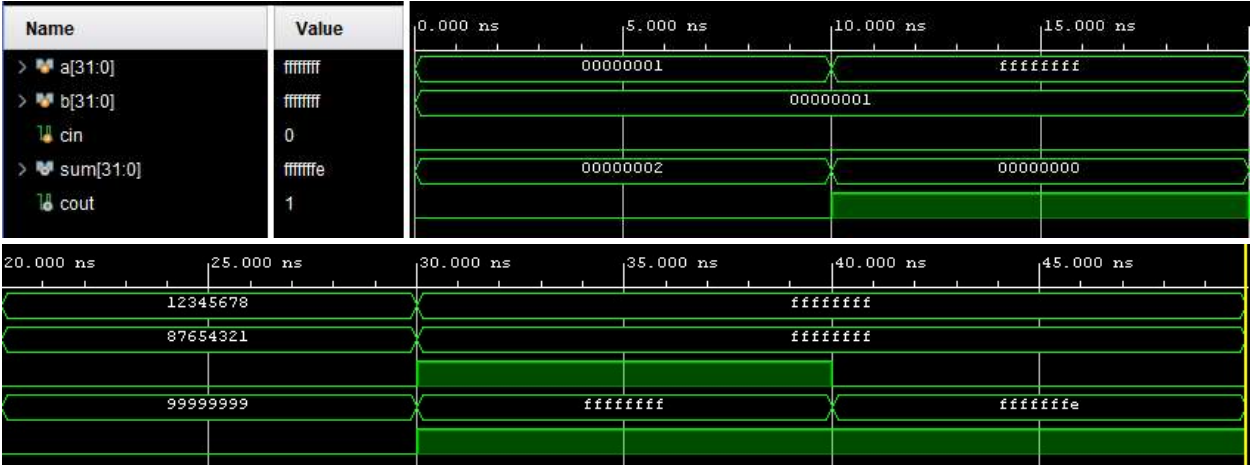
        $display("TB_PASS");
        $finish;
    end
end

```

```
initial #20000 begin $display("TB_FAIL: timeout"); $finish;
end

endmodule
```

Behavioral Simulation Results



Result: Issued test cases from testbench are successfully completed.

OpenLane RTL-to-GDS Flow

i. Synthesis

OpenLane reads the Verilog code, identifies the top-level ‘adder’ module, expands the generation loop, and maps the full-adder logic into physical standard-cell logic gates. In this example, the synthesized result contains repeated logic for XOR, AND, and OR operations.

61. Printing statistics.

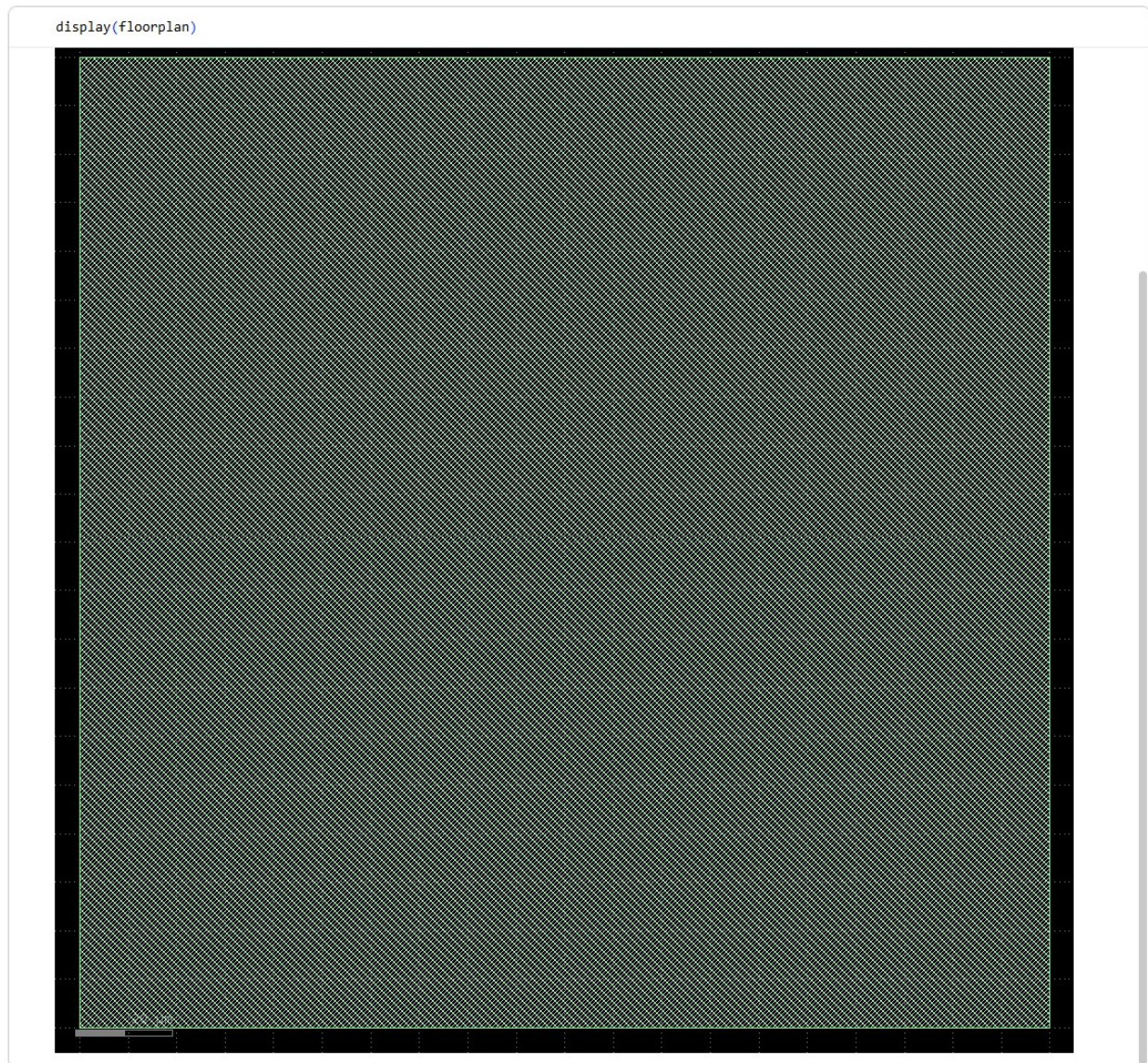
```
=== adder ===
```

```
Number of wires:           181
Number of wire bits:       274
Number of public wires:    37
Number of public wire bits: 130
Number of memories:        0
Number of memory bits:     0
Number of processes:       0
Number of cells:           209
sky130_fd_sc_hd__a21o_2    1
sky130_fd_sc_hd__a2bb2o_2  1
sky130_fd_sc_hd__a31o_2    2
sky130_fd_sc_hd__and2_2    2
sky130_fd_sc_hd__and2b_2   16
sky130_fd_sc_hd__and3_2    2
sky130_fd_sc_hd__buf_2     32
sky130_fd_sc_hd__nand2_2   33
sky130_fd_sc_hd__nand2b_2  12
sky130_fd_sc_hd__nor2_2    39
sky130_fd_sc_hd__o211a_2   1
sky130_fd_sc_hd__o21a_2    7
sky130_fd_sc_hd__o21ai_2   3
sky130_fd_sc_hd__o311a_2   13
sky130_fd_sc_hd__o31a_2    7
sky130_fd_sc_hd__or2_2     7
sky130_fd_sc_hd__or3_2     2
sky130_fd_sc_hd__xnor2_2   24
sky130_fd_sc_hd__xor2_2    5
```

```
Chip area for module '\adder': 1767.945600
```

ii. Floorplan

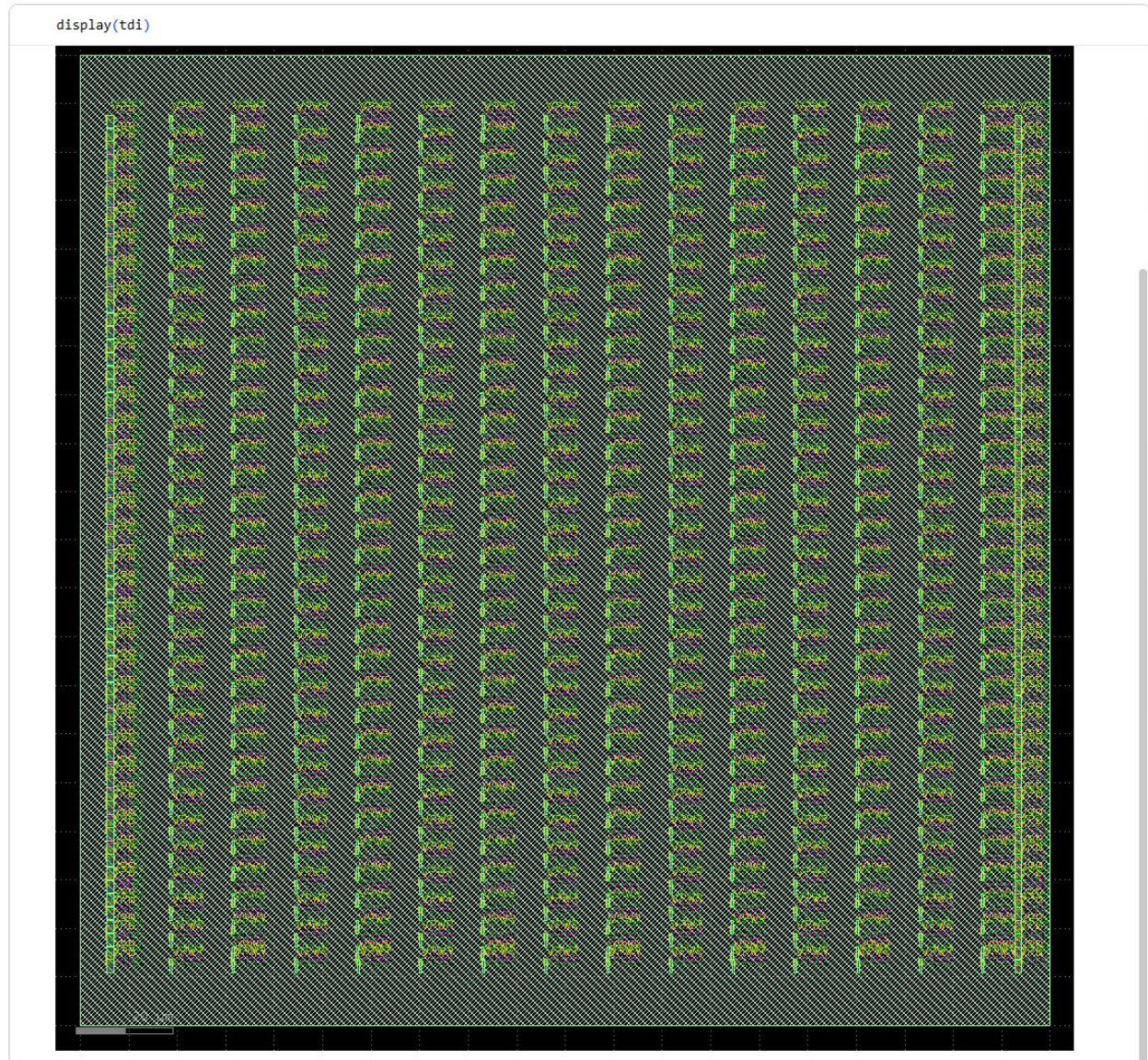
Floorplanning defined physical size and core area for the chip. I ran into an issue where the 32-bit adder consisted of 98 I/O pins, when there was only space for 92, so I had to define a larger die size by adding 'DIE_AREA=[0, 0, 200, 200],' in the floorplan.



iii. Tap/Endcap Cell Insertion

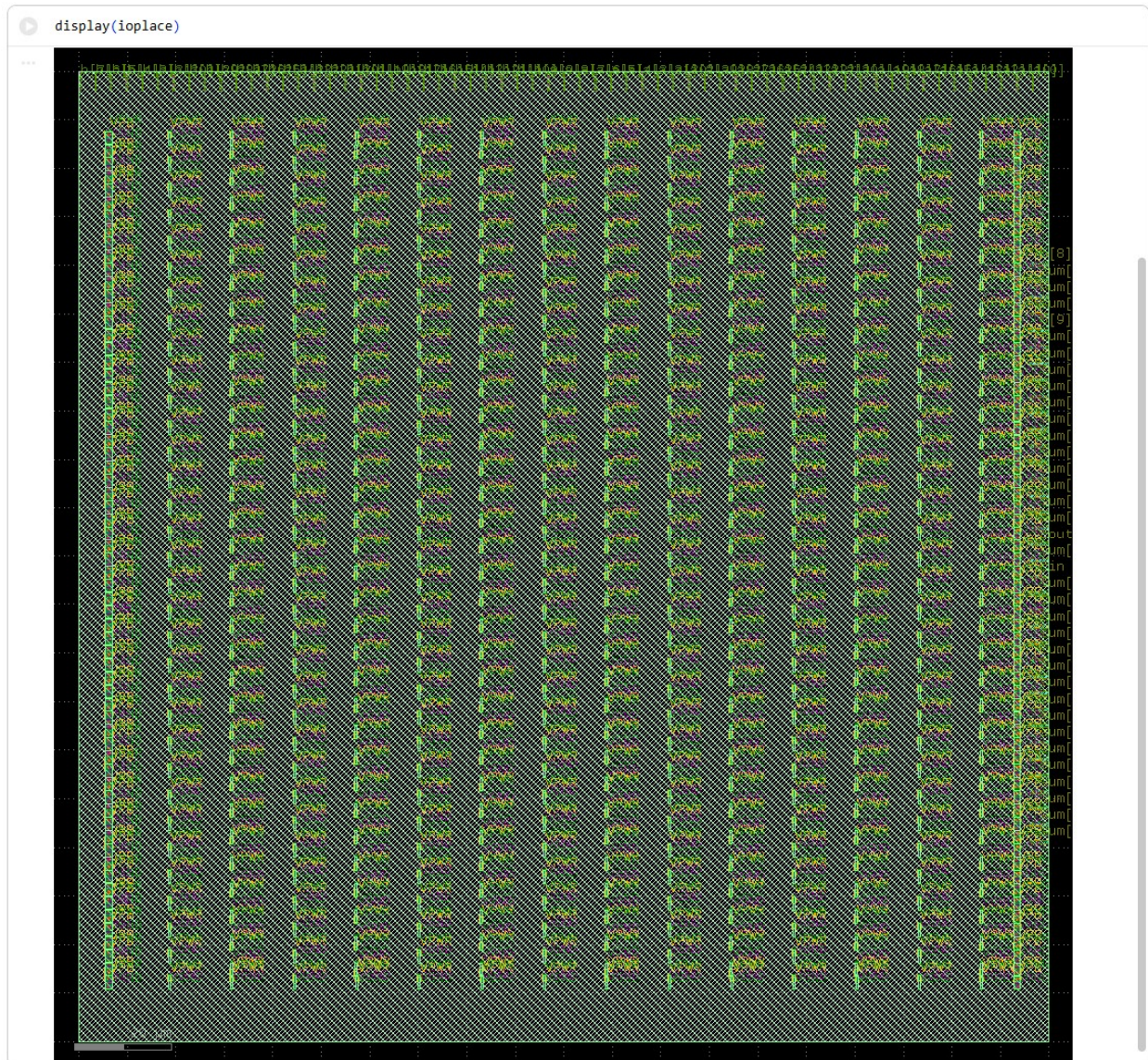
Tap Cells: help connect transistor bodies to proper power/ground

Endcap Cells: protect edges of standard-cell rows.



iv. I/O Placement

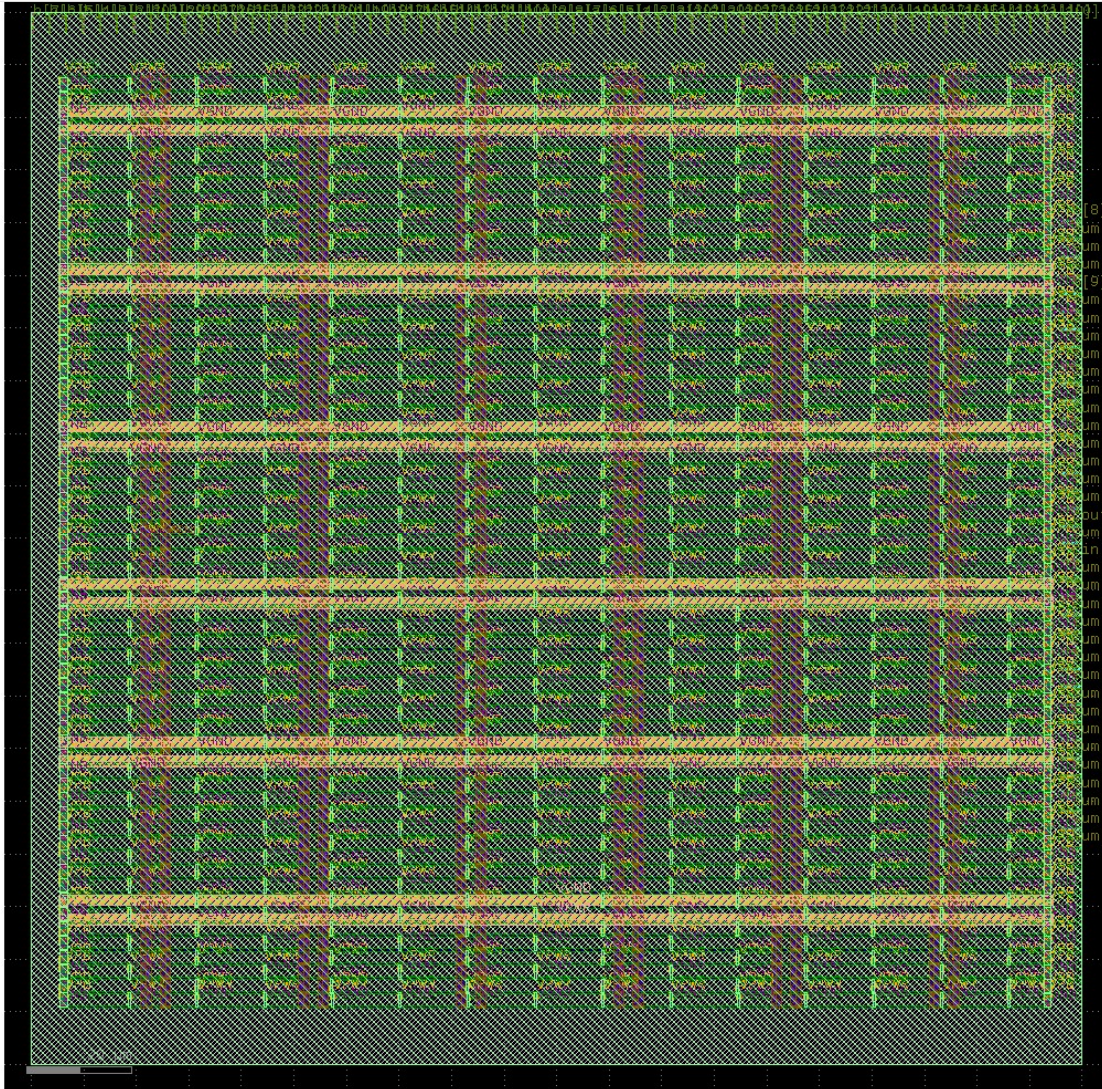
Assigns top-level input and output pins to locations around the die edge. There is a total of 98 pins needed (32 each for a, b, sum; 1 each for cin, cout), resulting in very dense placement around the perimeter.



v. Power Distribution Network

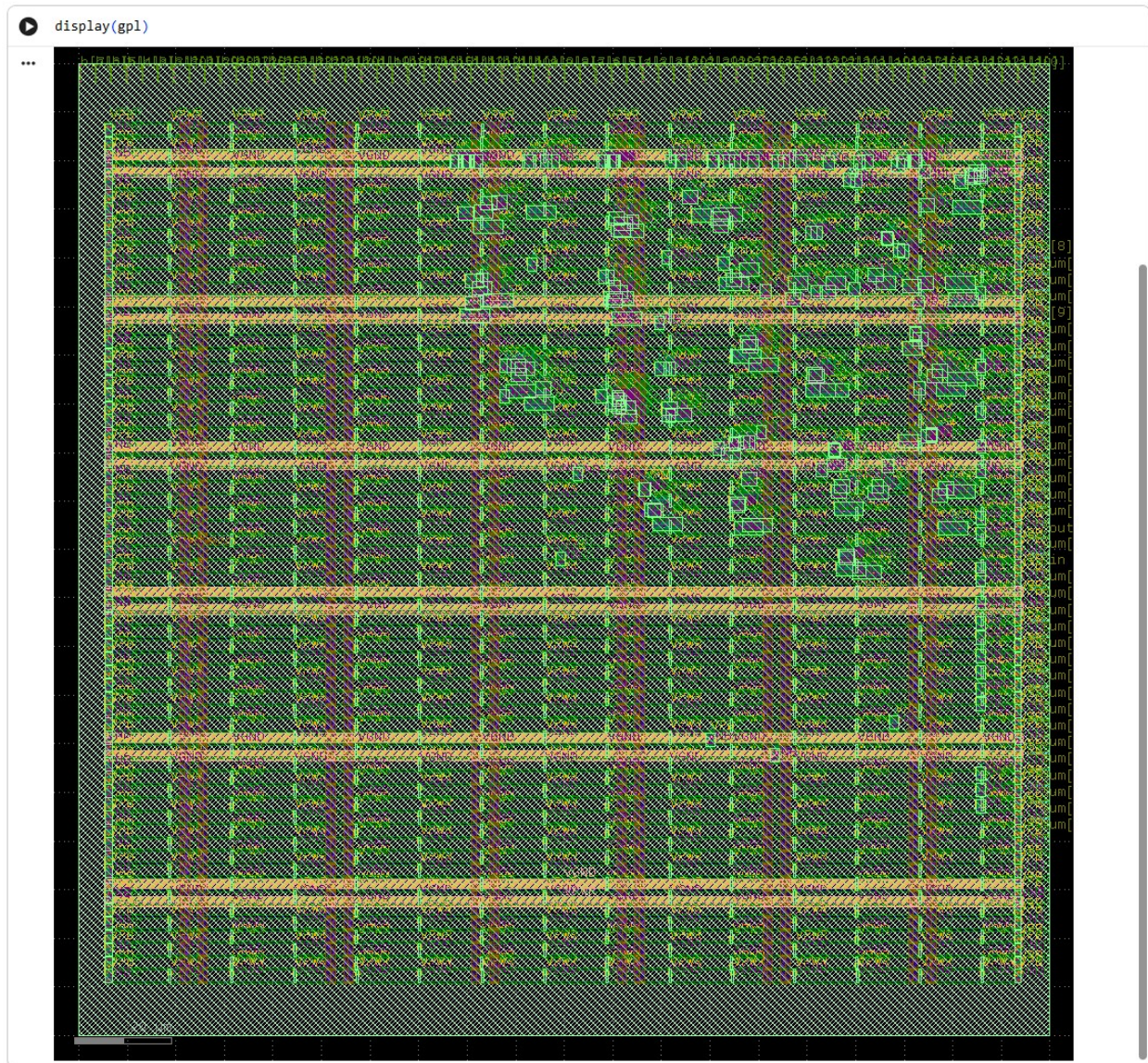
Adds power and ground routing structures across the chip.

```
display(pdn)
```



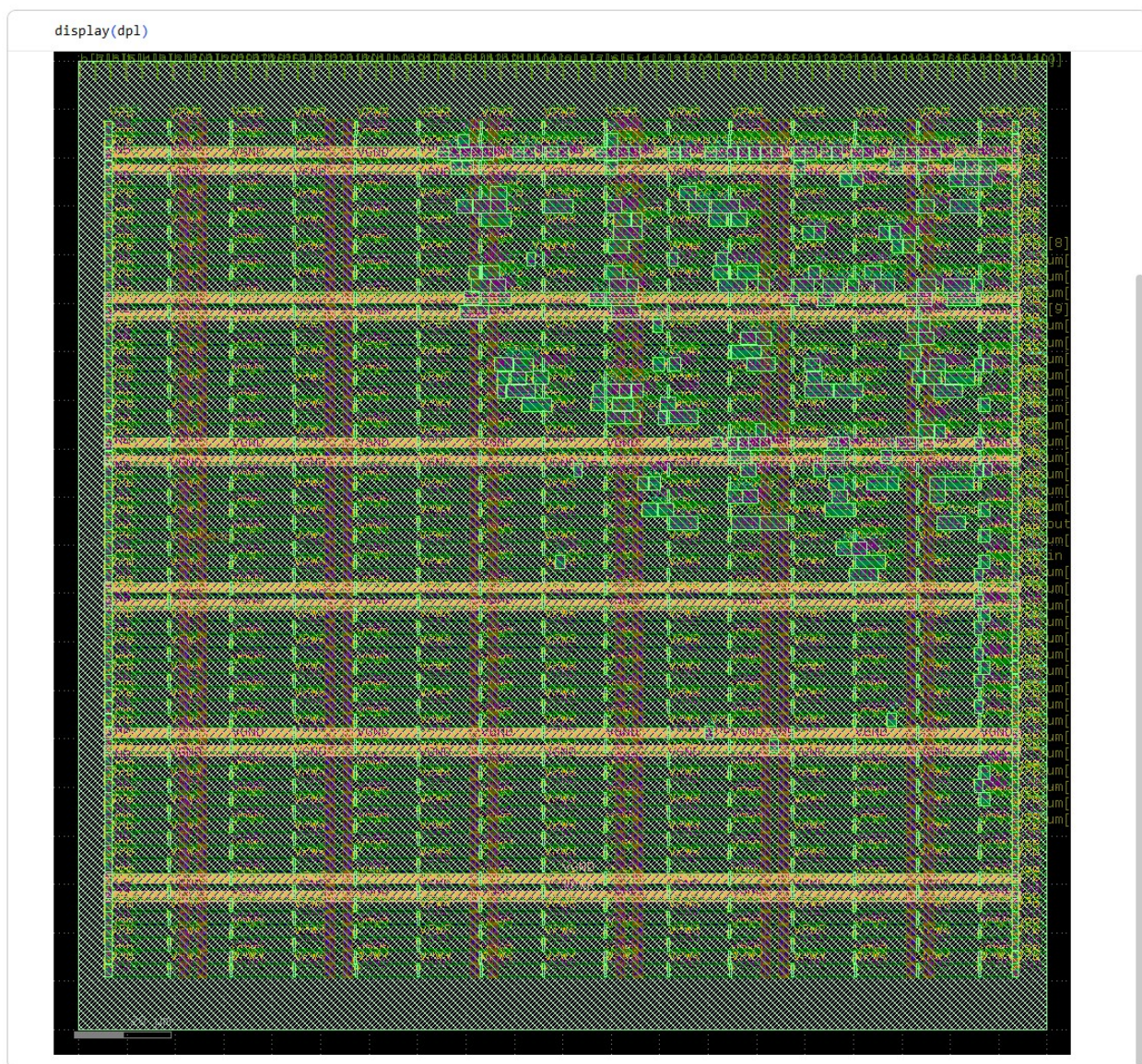
vi. Global Placement

Global Placement assigns approximate locations to cells inside the core area; cells are spread according to timing, congestion, and routability.

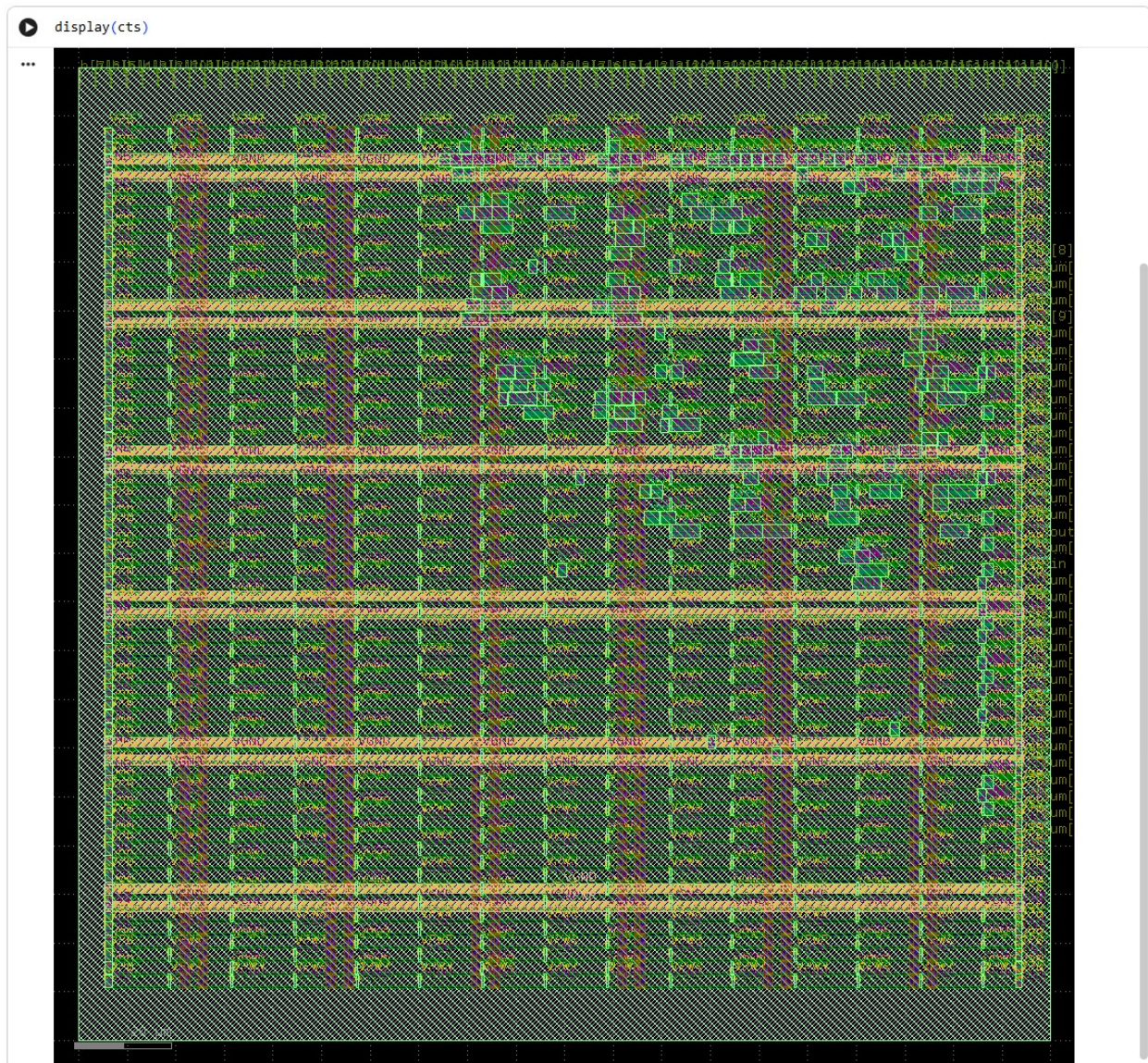


vii. Detailed Placement

Legalizes physical cell location.



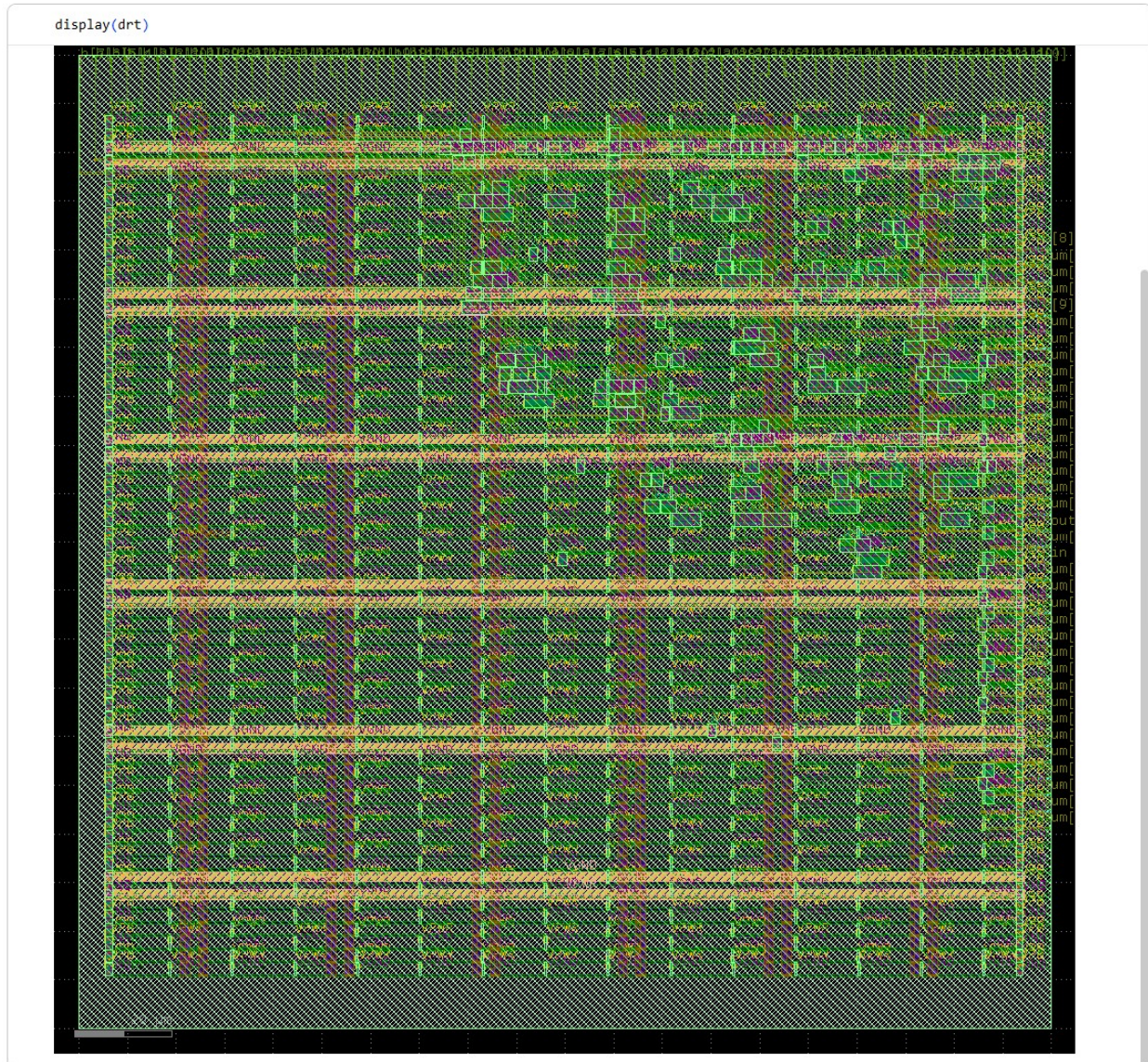
viii. Clock Tree Synthesis



As a combinational circuit, the clock tree synthesis is not inherently as significant as it is for sequential circuits.

ix. Detailed Routing

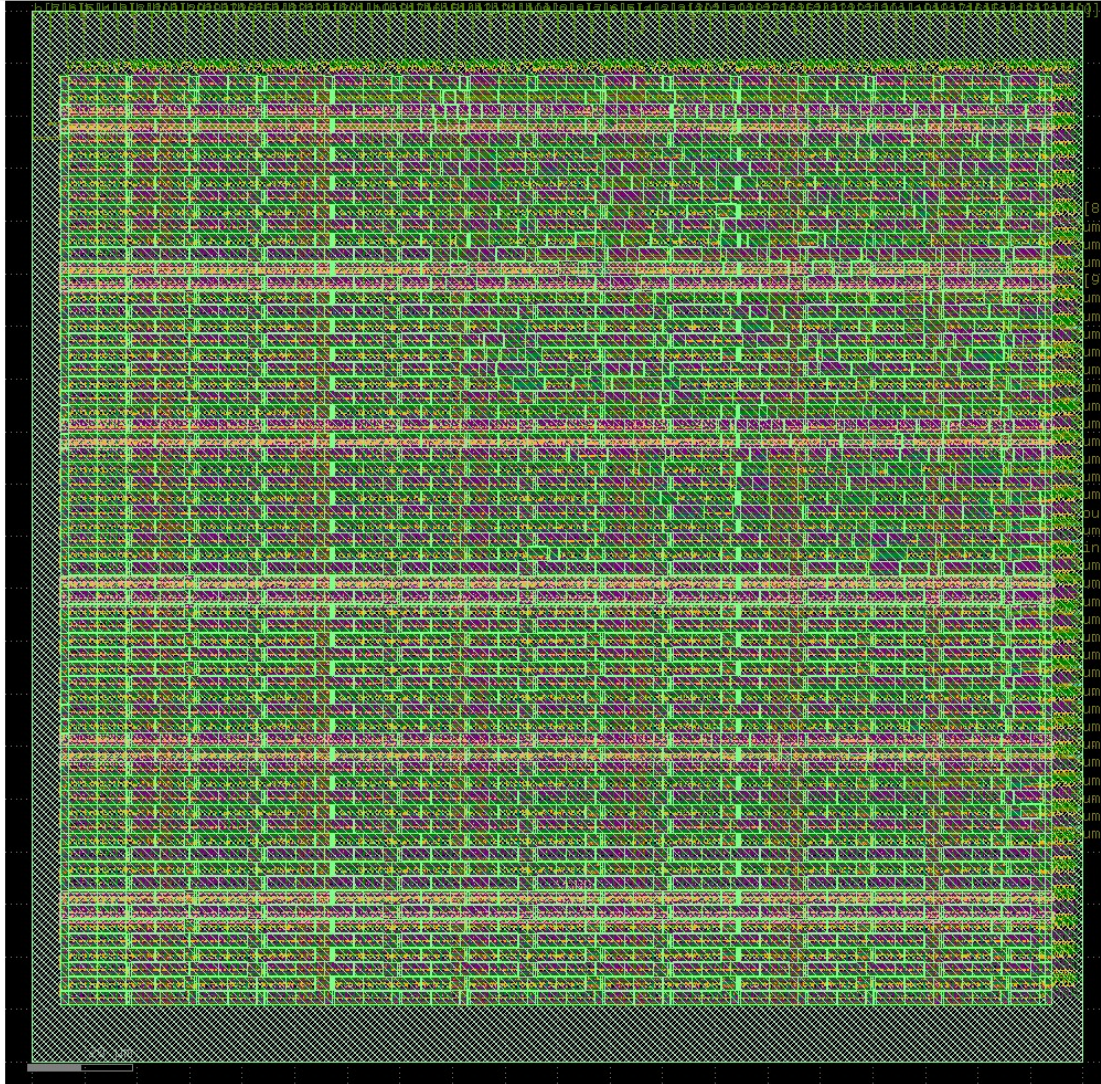
Completes physical wiring between cells and pins. Connects logic, carry chain, inputs, outputs, power, and ground using metal layers.



x. Fill insertion

Adds nonfunctional metal to satisfy layout density, resulting in a more manufacturable design.

```
display(fill)
```



xi. Static Timing Analysis

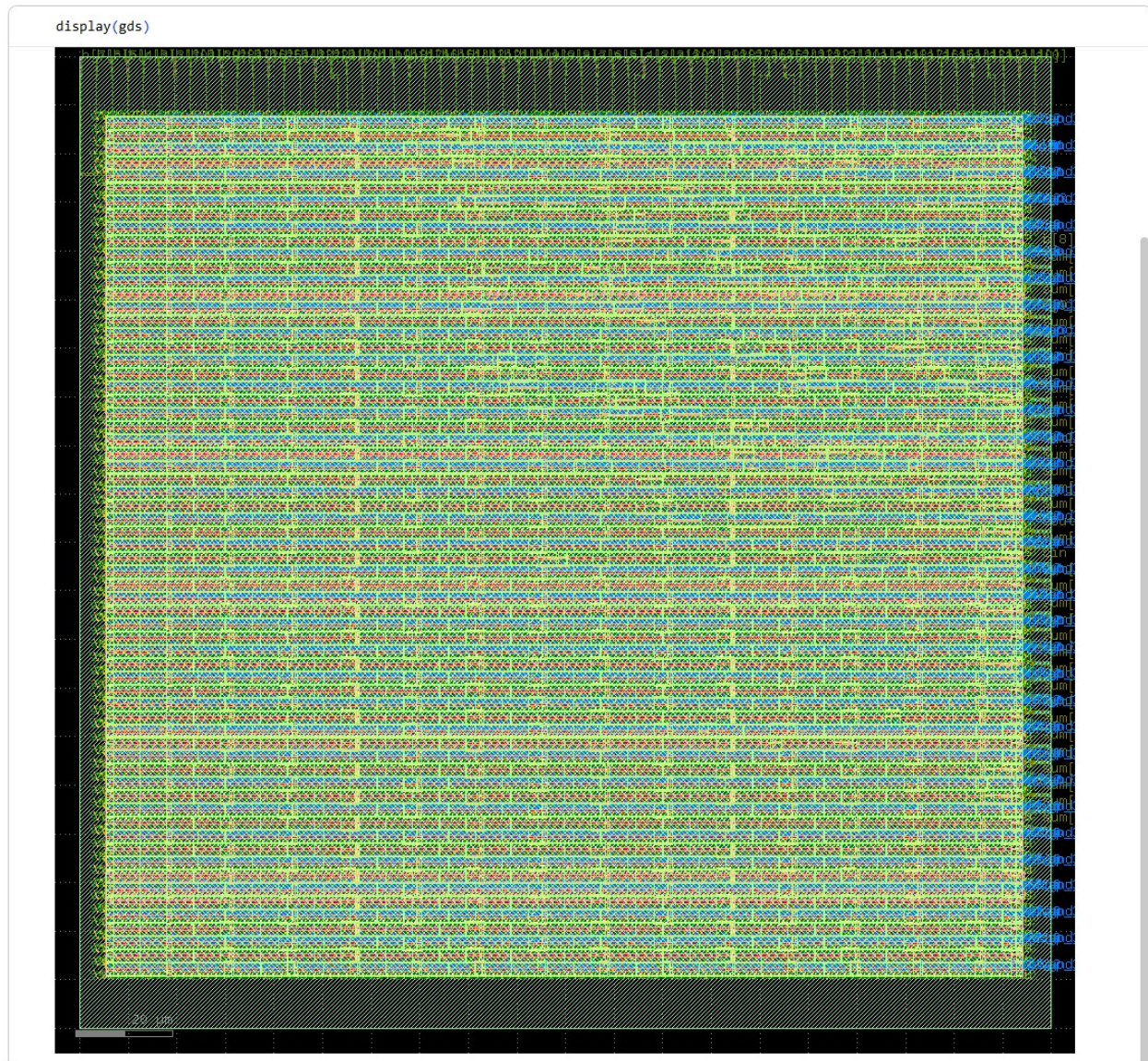
Used to evaluate timing behavior.

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Pat...	Setup TNS	Set... Vio Cou...	of which reg to reg	Max Cap Vio...	Max Slew Viol...
Overall	3.89...	N/A	0.0...	0	0	-6.4...	N/A	-73....	74	0	0	2
nom_tt_025C_1v80	3.99...	N/A	0.0...	0	0	-0.5...	N/A	-1.6...	4	0	0	0
nom_ss_100C_1v60	4.25...	N/A	0.0...	0	0	-6.4...	N/A	-72....	21	0	0	2
nom_ff_n40C_1v95	3.89...	N/A	0.0...	0	0	1.81...	N/A	0.00...	0	0	0	0
min_tt_025C_1v80	3.99...	N/A	0.0...	0	0	-0.5...	N/A	-1.4...	4	0	0	0
min_ss_100C_1v60	4.24...	N/A	0.0...	0	0	-6.3...	N/A	-70....	20	0	0	2
min_ff_n40C_1v95	3.89...	N/A	0.0...	0	0	1.84...	N/A	0.00...	0	0	0	0
max_tt_025C_1v80	4.00...	N/A	0.0...	0	0	-0.6...	N/A	-1.8...	4	0	0	0
max_ss_100C_1v60	4.26...	N/A	0.0...	0	0	-6.4...	N/A	-73....	21	0	0	2
max_ff_n40C_1v95	3.90...	N/A	0.0...	0	0	1.78...	N/A	0.00...	0	0	0	0

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violatio...	Max Slew Violati...
Overall	3.8946	N/A	0.0000	0	0	-6.4841	N/A	-73.3549	74	0	0	2
nom_tt_025C_1v80	3.9998	N/A	0.0000	0	0	-0.5947	N/A	-1.6443	4	0	0	0
nom_ss_100C_1v60	4.2535	N/A	0.0000	0	0	-6.4102	N/A	-72.0643	21	0	0	2
nom_ff_n40C_1v95	3.8981	N/A	0.0000	0	0	1.8123	N/A	0.0000	0	0	0	0
min_tt_025C_1v80	3.9943	N/A	0.0000	0	0	-0.5482	N/A	-1.4668	4	0	0	0
min_ss_100C_1v60	4.2429	N/A	0.0000	0	0	-6.3262	N/A	-70.6135	20	0	0	2
min_ff_n40C_1v95	3.8946	N/A	0.0000	0	0	1.8470	N/A	0.0000	0	0	0	0
max_tt_025C_1v80	4.0061	N/A	0.0000	0	0	-0.6362	N/A	-1.8008	4	0	0	0
max_ss_100C_1v60	4.2657	N/A	0.0000	0	0	-6.4841	N/A	-73.3549	21	0	0	2
max_ff_n40C_1v95	3.9026	N/A	0.0000	0	0	1.7809	N/A	0.0000	0	0	0	0

xii. GDS (Final Layout)

This format contains the standard cells, metal routing, vias, pins, power structures, and layout geometries required for chip fabrication; this is the completed physical implementation for the 32-bit adder.



Results & Discussion

- The OpenLane flow successfully transformed the Verilog RTL design into a physical GDS layout.
- The final layout matched the expected structure of a 32-bit ripple-carry adder, with repeated standard-cell patterns caused by the repeated full-adder stages.
- The design contained a large number of I/O pins because it used two 32-bit inputs, one 32-bit output, cin, and cout.
- The dense I/O placement around the die perimeter was expected due to the 98 total top-level pins.
- The die/core area had to be adjusted during floorplanning to provide enough space for I/O placement and routing.
- The routed layout showed that OpenLane was able to connect the internal logic, carry chain, input pins, output pins, and power network.
- The ripple-carry adder's main timing limitation is the carry chain, since each bit must wait for the carry from the previous full-adder stage.
- The expected critical path is from the least significant carry input through the full-adder carry chain toward the most significant carry output.
- Since the design is combinational, clock-related warnings were expected and did not prevent the physical implementation from being generated.
- The layout was not optimized for minimum area; the main goal was to complete and observe the full RTL-to-GDS flow.
- The final GDS result confirms that the 32-bit adder was physically implemented using the SKY130 standard-cell library.
- Overall, the results show that the design was functionally correct in simulation and physically realizable through OpenLane.

Conclusion

In this project, a 32-bit ripple-carry adder was successfully designed, verified, and physically implemented using Verilog HDL and OpenLane. The Verilog implementation used a modular structure with a 1-bit full-adder module instantiated repeatedly through a generate loop. This made the design scalable and easy to understand, while also clearly showing the carry chain behavior that defines a ripple-carry adder.

Functional verification confirmed that the adder produced the correct sum and carry-out values for multiple test cases, including cases involving overflow and maximum 32-bit input values. After verification, the design was taken through the OpenLane RTL-to-GDS flow. The flow successfully converted the RTL code into a standard-cell-based physical layout using the SKY130 process. Important physical design steps included synthesis, floorplanning, I/O placement, power distribution, placement, routing, static timing analysis, and final GDS generation.

One important challenge encountered during the project was the large number of I/O pins required by the 32-bit design. Since the adder uses two 32-bit inputs, one 32-bit output, cin, and cout, the design required 98 top-level pins. This caused an I/O placement issue, which was solved by increasing the die area during floorplanning. The final layout also showed repeated standard-cell patterns, which matched the repeated full-adder structure of the ripple-carry adder.

Overall, this project showed the complete transition from RTL design to physical chip layout. It strengthened understanding of Verilog design, functional simulation, standard-cell synthesis, physical layout generation, and the practical challenges involved in digital ASIC design. The completed GDS layout demonstrates that the 32-bit adder was not only functionally correct in simulation, but also physically realizable through the OpenLane design flow.